arXiv:2611.12732v1 [cs.LG] 9 Nov 2026

# Σ-LeWM: Learning World Models via S-Information for Robot Control

**Xinrui Zu** **Michael Przystupa** **Zhao Yang** **Shujian Yu** **Kevin Sebastian Luck**

Vrije Universiteit Amsterdam, The Netherlands

Website Code Hugging Face

## ABSTRACT

World models have increasingly been applied as a model-based control method for robotics tasks such as manipulation and locomotion. While early world model architectures relied on decoders to prevent mode collapse and learn a predictable latent space, recent works on world models, such as Joint Embedding Predictive Architecture or LeWorldModel, increasingly aim to learn embeddings without a grounding decoder, utilising only a latent forward dynamics predictor and a stabilising entropy term. In this paper, we set out to demonstrate that the further introduction of two additional latent losses to LeWorldModel, a backward dynamics predictor and an inverse dynamics action predictor, enforces further structure on the latent embedding, leading to further gains in both performance and planning efficiency. These are critical properties for improving the applicability of such methods to real-time model-based control of real-world robots. We demonstrate these gains on standard world model benchmarks and two simulated robot manipulation tasks.

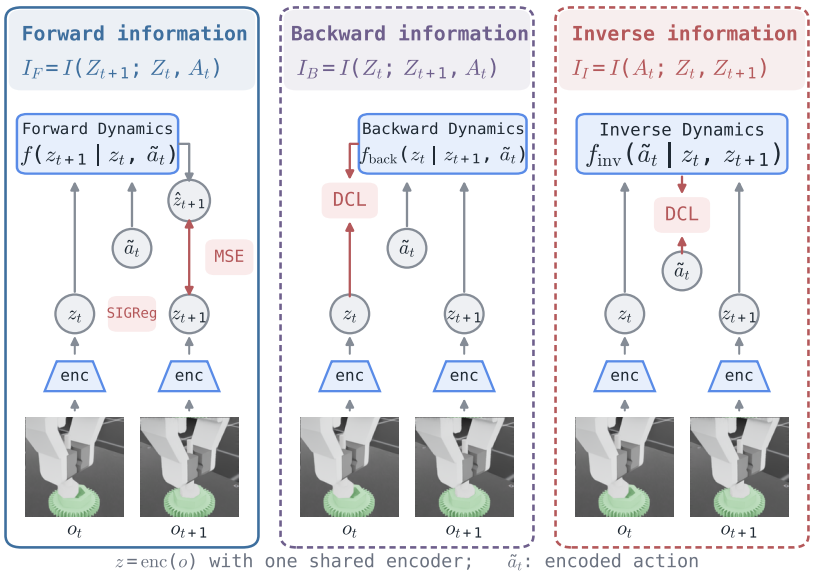



Figure 1: Σ-LeWM introduces two auxiliary tasks for the latent dynamics learning in LeWorldModel (LeWM): Next to learning the standard forward dynamics function in the latent space (left), Σ-LeWM learns also implicitly a backward dynamics (middle) and inverse dynamics function (right). Learning these functions alongside standard LeWM imposes further restrictions and biases on the latent embedding space. At test time actions are found by standard forward planning with the forward dynamics (left) and model-predictive control (MPC) algorithms.

# 1 Introduction

World models have become increasingly useful for model-based control of robots operating in complex observation spaces such as images or high-dimensional proprioceptive data [1–5]. Instead of planning directly in complex high-dimensional spaces, the core idea is to learn an embedding space of observations and perform planning in a dimensionally reduced space efficiently and quickly [6]. This requires an embedding function that projects high-dimensional observations into latent features where a transition function can be learned to perform model-predictive control. In this work, we investigate ways to improve world model learning by incorporating multiple learning signals during optimisation.

Early world models have relied heavily on encoder-decoder architectures, such as variational autoencoders [7], to prevent mode collapse and ensure reliable learning for dynamics learning and planning. The decoder's prediction of the original observation typically imposes artificial restrictions on the learned latent space: each observation feature must be fully predictable to satisfy and minimise the reconstruction loss. Instead, in the context of planning and model-based control, our main objective is a structured latent space that places similar observations close together, while maximising the ability of a learned transition function to predict the next latent state given an action.

As prior work has shown, removing the decoder, while challenging, yields better-structured latent spaces and improved planability, leading to performance gains in robot control tasks [8]. Notably, recent work such as LeWorldModel (LeWM) [9] shows that using only a latent transition loss that predicts the next latent features, along with a Sketched-Isotropic-Gaussian Regularizer to prevent mode collapse in latent representations, enables training world models applicable to robotic tasks such as object manipulation. In this work, we go one step further: we show a natural and intuitive extension to LeWorldModel, motivated by insights from information theory, in which an additional backward transition-dynamics loss and an inverse dynamics loss induce further structure on the world model's latent embedding space. We show that this leads not only to better performance, but also to computational efficiency by allowing reductions in both horizon length and inference steps, which lowers the test-time planning compute of the model-predictive control process and better real-world applicability in robotics.

In summary, the contributions of our paper are:

- C1 We propose a new extension of LeWM, namely Σ-LeWM, which introduces backward and inverse dynamics as auxiliary losses/tasks to learn a more structured & predictable latent embedding.
- C2 We show that this approach has an intuitive grounding in information theory, where this learning approach is motivated by maximising the S-Information.
- C3 Our experiments demonstrate that Σ-LeWM leads to better performance than standard LeWM, and that it also does so on more realistic robotics tasks such as gear assembly.

# 2 Related Works

**Latent World Models and JEPA.** Generative world models like the Dreamer family [6], PlaNet [10], and DIAMOND [11] learn latent dynamics through reconstruction with decoders and reward signals. TD-MPC [8], instead, drops the decoder and learns the latent dynamics via temporal-difference objectives. Recently, JEPA [12] has gained increasing attention with no need for reconstruction or reward, and learns to predict latent representations using an encoder and a predictor. Methods like I-JEPA [12] and V-JEPA [13] use JEPA for self-supervised representation learning by predicting the latent representation of masked inputs. However, latent prediction does not guarantee informative representations and can suffer from representational collapse.

**Anti-collapse in Latent World Models.** Recent JEPA-like methods use various approaches to alleviate representation collapse and stabilise training: DINO-WM [14] freezes a pretrained encoder to prevent the prediction from collapsing; while PLDM [15], Sub-JEPA [16], and RC-aux [17] use auxiliary objectives to regularise the latent space. However, preventing collapse alone does not guarantee that the latent space captures control-relevant structure, which is useful in robot learning and planning.

**Inverse Dynamics in Latent World Models.** Recent works increasingly use inverse dynamics to make latent representations more control-relevant. SMWM [18] and AC-MTM [19] use inverse action prediction from the two latent representations as an auxiliary task when learning representations. On the other hand, GC-IDM [20] and ACID [21] use inverse dynamics only for downstream planning, without changing the latent representation learning.

# 3 Preliminaries

## 3.1 LeWorldModel

We assume a decoder-less latent world model consisting of

$$\text{an Encoder } E : o_t \mapsto z_t, \tag{1}$$

$$\text{latent transition function } f : z_t, a_t \mapsto z_{t+1}, \tag{2}$$

where the variables $z_t$ and $z_{t+1}$ represent latent encodings of observations $o_t$ and $o_{t+1}$ respectively. Observations $o_t$ are often assumed to be high-dimensional, such as images. In LeWM, the functions given above are trained with the two objectives combining the latent next-state prediction term $\mathcal{L}_{\text{pred}}$ and a latent state regularisation term $\mathcal{L}_{\text{reg}}$, i.e.,

$$\mathcal{L}_{\text{LeWM}} = \underbrace{\|f(z_t, a_t) - z_{t+1}\|_2^2}_{\mathcal{L}_{\text{pred}}} + \lambda_{\text{sig}} \underbrace{\mathcal{L}_{\text{SIGReg}}(Z)}_{\mathcal{L}_{\text{reg}}}, \tag{3}$$

where SIGReg is a regularising term introduced by LeWM to prevent mode collapse of the latent embedding in the absence of a decoder.

At test time, a planning algorithm in the latent space of the world model computes the actions. Given the representation of the current state $E(o_0) = z_0$ and an encoded goal state $E(o_{\text{goal}}) = z_{\text{goal}}$, an MPC-like planning method samples candidate action sequences and rolls them forward in the latent embedding space. Thus, the planner optimises the objective

$$a^{\star}_{0:H-1} = \arg\min_{a_{0:H-1}} \|h(z_0, a_{0:H-1}, f) - z_{\text{goal}}\|_2^2, \tag{4}$$

where $h(z_0, a_{0:H-1}, f) = \tilde{z}_H$ is the predicted final latent state of a latent trajectory with start state $z_0$ and rolled forward using the learned transition function $z_{t+1} = f(z_t, a_t)$. Following LeWM, we also use the cross-entropy method (CEM), a sampling-based optimiser that iteratively selects candidate action sequences according to their latent-space goal cost, to find the optimal trajectory. Once an optimal latent trajectory is found, the first action(s) is executed on the system before the next planning cycle starts.

## 3.2 Decoupled Contrastive Information Maximisation

To optimise the backward and inverse predictive objectives, we adopt the decoupled contrastive loss (DCL) [22]. DCL can be viewed as a contrastive surrogate for mutual information maximisation, closely related to the InfoNCE lower bound [23, 24].

For a batch of paired representations $\{(q_i, k_i)\}_{i=1}^N$, we define the normalised similarity $s_{ij} = \frac{\langle \hat{q}_i, \hat{k}_j \rangle}{\tau}$, where $\hat{q}_i = q_i/\|q_i\|_2$ and $\hat{k}_j = k_j/\|k_j\|_2$ are the normalised critic outputs, and $\tau$ is a temperature parameter.

The directional DCL objective is defined as:

$$\mathcal{L}_{\text{DCL}}^{q\to k} = \frac{1}{N} \sum_i \left[ -s_{ii} + \log \sum_{j \neq i} \exp(s_{ij}) \right]. \tag{5}$$

We use its symmetric form

$$\mathcal{D}(Q, K) = \frac{1}{2} \left( \mathcal{L}_{\text{DCL}}^{q\to k} + \mathcal{L}_{\text{DCL}}^{k\to q} \right). \tag{6}$$

Unlike InfoNCE, DCL removes the positive pair from the normalisation term, thereby decoupling positive attraction from negative-sample repulsion. For each sample, the corresponding InfoNCE loss satisfies:

$$\ell_{\text{InfoNCE}} = \log\left(1 + \exp(\ell_{\text{DCL}})\right), \tag{7}$$

which is a strictly increasing function of $\ell_{\text{DCL}}$. Therefore, minimising DCL is consistent with maximising the corresponding InfoNCE mutual-information lower bound.

# 4 Methodology

In this section, we first introduce our proposed Σ-LeWM, an extension of the standard LeWM, and then provide formal intuition for the effects of the proposed architecture.

### 4.1 Overview of Σ-LeWM

We extend the standard latent world model, specifically LeWM, with two additional predictive functions: the backward and inverse transition functions.

**Components:** Therefore, our proposed extension to LeWM contains the following main components:

$$\text{Observation encoder:} \quad \mathbf{z_t} = E^O(o_t) \tag{8a}$$

$$\text{Action encoder:} \quad \tilde{\mathbf{a}}_t = E^A(a_t) \tag{8b}$$

$$\text{Forward Dynamics:} \quad \mathbf{z}_{t+1} = f(\mathbf{z}_t, \tilde{\mathbf{a}}_t) \tag{8c}$$

$$\text{Backward Dynamics (aux.):} \quad \mathbf{z}_t = f_{\text{back}}(\mathbf{z}_{t+1}, \tilde{\mathbf{a}}_t) \tag{8d}$$

$$\text{Inverse Dynamics (aux.):} \quad \tilde{\mathbf{a}}_t = f_{\text{inv}}(\mathbf{z}_t, \mathbf{z}_{t+1}) \tag{8e}$$

Building on LeWM, Σ-LeWM retains the standard observation and action encoders $E(\cdot)$ and $E^A(\cdot)$, together with the forward dynamics model $f(\mathbf{z}_t, \tilde{\mathbf{a}}_t)$. These auxiliary objectives ($f_{\text{back.}}(\cdot)$ and $f_{\text{inv.}}(\cdot)$) are used only during training and do not introduce an additional dynamic model at inference time. They can be trained implicitly by embedding them directly in the DCL loss (see Sec. 4.2). Consequently, test-time planning remains identical to LeWM. Nevertheless, by maximising the backward and inverse information objectives during training, Σ-LeWM imposes additional structural biases on the learned latent space.

### 4.2 Training

To train the individual functions of our world model in Eq (8), we use an objective composed of three individual loss terms, namely

$$\mathcal{L}_\Sigma = \mathcal{L}_{\text{forward}} + \lambda_{\text{back.}}\mathcal{L}_{\text{back.}} + \lambda_{\text{inv.}}\mathcal{L}_{\text{inv.}}, \tag{9}$$

where $\mathcal{L}_{\text{forward}}$ is the forward world-model objective and $\mathcal{L}_{\text{back.}}, \mathcal{L}_{\text{inv.}}$ optimise the backward and inverse views, respectively. $\lambda_{\text{back}}$ and $\lambda_{\text{inv.}}$ are regularisation weights.

**Forward Loss ($\mathcal{L}_{\text{forward}}$):** The forward network is trained with the same loss functions as LeWM plus a multi-step prediction term with weight $\lambda_{\text{multi}} = 1$, i.e., $\mathcal{L}_{\text{forward}} = \mathcal{L}_{\text{LeWM}} + \sum_{k=2}^{m} \|\hat{z}_{t+k} - z_{t+k}\|_2^2$, together with LeWM's latent space regularisation term.

**Backward Loss ($\mathcal{L}_{\text{back}}$):** To train the backward dynamics function $\mathbf{z}_t = f_{\text{back}}(\mathbf{z}_{t+1}, \tilde{\mathbf{a}}_t)$ we utilise the DCL as introduced in Eq. (6). Given a transition $(o_t, a_t, o_{t+1})$ we define the backward loss as:

$$\mathcal{L}_{\text{back.}} = \mathcal{D}(f_{\text{back.}}(z_{t+1}, \tilde{a}_t), z_t) \tag{10}$$

where $z_{t+1} = E(o_{t+1})$, $\tilde{a}_t = E^A(a_t)$ and $z_t = E(o_t)$ with the goal of aligning the representation of the future state-action pair $(z_{t+1}, \tilde{a}_t)$ with the latent representation $z_t$ of the current observation $o_t$. In our experiments, we found that training an additional encoder in the representation $z_t$ improved performance; therefore, in our implementations, we optimise $\mathcal{L}_{\text{back.}} = D(f_{\text{back.}}(z_{t+1}, \tilde{a}_t), E^z(z_t))$.

**Inverse Loss ($\mathcal{L}_{\text{inv.}}$):** In a similar fashion to the backward loss, we utilise the DCL loss in Eq. (6) again for training the inverse dynamics function with:

$$\mathcal{L}_{\text{inv.}} = \mathcal{D}(f_{\text{inv.}}(z_t, z_{t+1}), \tilde{a}_t). \tag{11}$$

This objective maximises the inverse information dependence between the action representation $\tilde{a}_t$ and the pair of consecutive latent states $(z_t, z_{t+1})$.

### 4.3 Theoretical Analysis: An Information-Theoretic Intuition

An information-theoretic view of the above learning objectives provides a unified interpretation of the forward, backward, and inverse objectives through multivariate information maximisation.

For the following discussion, we will use $Z_t, Z_{t+1}$ and $\tilde{A}_t$ for readability and conciseness, but note that these are direct encodings of the original observations $O_t, O_{t+1}$ and actions $A_t$. The objective of LeWM in Eq. (3) contains two terms. Regarding the prediction term, let us assume a fixed-variance Gaussian predictive model:

$$p_\phi(Z_{t+1} \mid Z_t, A_t) = \mathcal{N}\left(g_\phi(Z_t, A_t), \sigma^2 I\right). \tag{12}$$

We have:

$$-\log p_\phi(Z_{t+1} \mid Z_t, A_t) = \frac{1}{2\sigma^2}\|Z_{t+1} - g_\phi(Z_t, A_t)\|_2^2 + \frac{d}{2}\log(2\pi\sigma^2), \tag{13}$$

hence minimising the prediction MSE is equivalent, up to an additive constant, to minimising the conditional negative log-likelihood, which can be further decomposed as:

$$\begin{aligned}
&\mathbb{E}_{p(Z_t,A_t,Z_{t+1})}[-\log p_\phi(Z_{t+1} \mid Z_t, A_t)] \\
&= \mathbb{E}_{p(Z_t,A_t,Z_{t+1})}[-\log p(Z_{t+1} \mid Z_t, A_t)] \\
&\quad + \mathbb{E}_{p(Z_t,A_t,Z_{t+1})}\left[\log \frac{p(Z_{t+1} \mid Z_t, A_t)}{p_\phi(Z_{t+1} \mid Z_t, A_t)}\right] \\
&= H(Z_{t+1} \mid Z_t, A_t) \\
&\quad + \mathbb{E}_{p(Z_t,A_t)} D_{\mathrm{KL}}(p(Z_{t+1} \mid Z_t, A_t) \| p_\phi(Z_{t+1} \mid Z_t, A_t)) \,.
\end{aligned} \tag{14}$$

Since the KL divergence is non-negative, the conditional negative log-likelihood upper-bounds $H(Z_{t+1} \mid Z_t, A_t)$. Therefore, minimising the predictive MSE minimises a variational upper bound on the conditional entropy.

Second, SIGReg encourages the aggregated latent distribution to match an isotropic Gaussian $q(Z_{t+1}) \approx \mathcal{N}(0, I)$. It is well known that, among all multivariate continuous distributions with a fixed covariance matrix, the Gaussian distribution maximises the Shannon differential entropy. Hence, SIGReg also maintains a high marginal entropy $H(Z_{t+1})$.

Combining these two effects, we can directly relate the objectives of the original LeWM as follows:

$$\underbrace{\mathcal{L}_{\mathrm{pred}}}_{\downarrow H(Z_{t+1}|Z_t,A_t)} + \lambda \underbrace{\mathcal{L}_{\mathrm{SIGReg}}}_{\uparrow H(Z_{t+1})} . \tag{15}$$

and since

$$I(Z_{t+1}; Z_t, A_t) = H(Z_{t+1}) - H(Z_{t+1} \mid Z_t, A_t), \tag{16}$$

we arrive at the insight that the two objectives jointly optimise

$$\max \; I(Z_{t+1}; Z_t, A_t). \tag{17}$$

Therefore, LeWM can be interpreted as a form of *joint mutual information maximisation*.

Σ-LeWM's main modification lies in the backward and inverse directions: it optimises backward and inverse objectives directly through DCL, encouraging high mutual information between $Z_t$ and $(Z_{t+1}, \tilde{A}_t)$, and between $\tilde{A}_t$ and $(Z_t, Z_{t+1})$, respectively.

In a similar fashion, the backward and inverse objectives directly maximise contrastive lower bounds on the corresponding mutual information terms:

$$\mathcal{I}_{\mathrm{back}} = I(Z_t; Z_{t+1}, \widetilde{A}_t), \tag{18a}$$

$$\mathcal{I}_{\mathrm{inv}} = I(\widetilde{A}_t; Z_t, Z_{t+1}). \tag{18b}$$

Specifically, we optimise both terms using DCL, which is monotonically related to the corresponding InfoNCE objective and therefore serves as a surrogate for maximising its mutual information lower bound.

Notably, we set out to optimise the sum of the three joint mutual information terms given above. It has been shown that such a sum directly relates to the Total Correlation (TC) [25] and Dual Total Correlation (DTC) [26], i.e.

$$\begin{aligned}
&\mathcal{I}_{\mathrm{forward}} + \mathcal{I}_{\mathrm{back.}} + \mathcal{I}_{\mathrm{inv.}} \\
&= \mathrm{TC}(Z_t, \tilde{A}_t, Z_{t+1}) + \mathrm{DTC}(Z_t, \tilde{A}_t, Z_{t+1}).
\end{aligned} \tag{19}$$

The decomposition in Eq. (19) provides a useful information-theoretic interpretation of the proposed objective. TC quantifies the overall statistical dependence among $Z_t$, $\tilde{A}_t$, and $Z_{t+1}$, vanishing only when the three variables are mutually independent. DTC, by contrast, captures a complementary form of multivariate dependence that reflects how strongly each variable is constrained by the remaining variables. More importantly, their sum admits a particularly intuitive form:

$$\Sigma(Z_t, \tilde{A}_t, Z_{t+1}) = \mathrm{TC}(Z_t, \tilde{A}_t, Z_{t+1}) + \mathrm{DTC}(Z_t, \tilde{A}_t, Z_{t+1}), \tag{20}$$

which is known as the S-Information [27].

Thus, rather than encouraging only forward predictability, our objective promotes predictive dependence in all three directions: the next state from the current state and action, the current state from the next state and action, and the action from the two consecutive states. We therefore refer to the proposed method as Σ-LeWM.

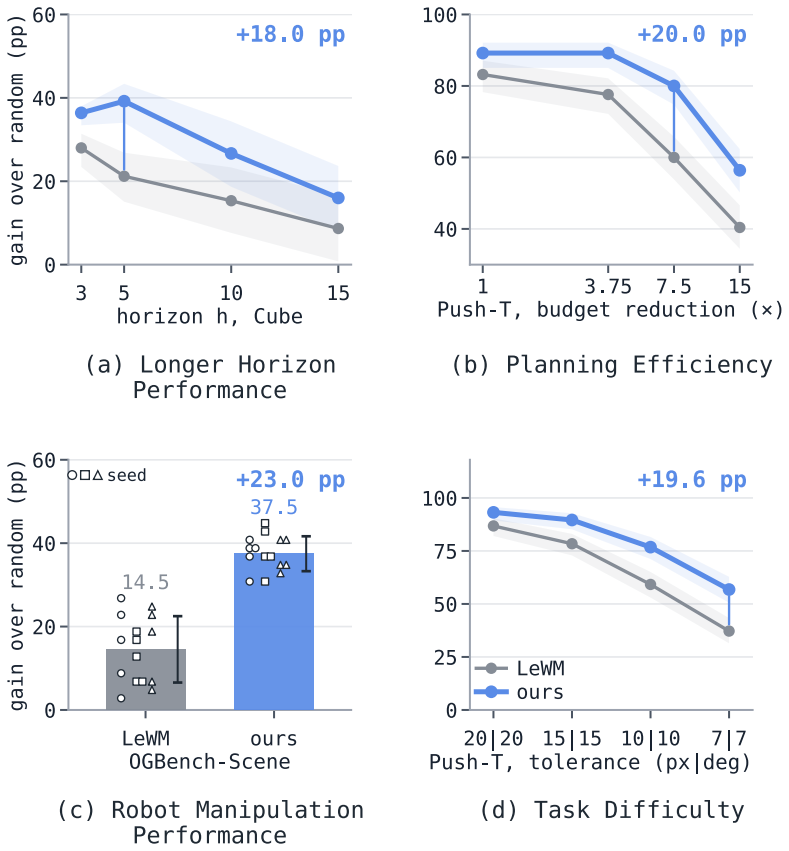



Figure 2: Overview of main experimental results: Σ-LeWM outperforms standard LeWM when considering long horizon tasks (a), and leads to improved planning efficiency (b). Both properties are important for complex and realistic robotic tasks. We can demonstrate superior performance over LeWM on the OGBench-Scene manipulation task (c; gain over the random-action floor; points: individual evaluations, marker shape: training seed, error bar: std over all 15 evaluations as in Table 1) and demonstrate better performance of Σ-LeWM when considering high-accuracy tasks in robotics (d).

## 5 Experiments

In our experiments, we set out to investigate whether (RQ1) the proposed Σ-LeWM shows performance gains over LeWM, (RQ2) what effect the auxiliary tasks have on the latent embedding and its structure, and, finally, (RQ3) if maximising the S-Information provides an advantage over maximising the individual terms.

To answer the first question, we evaluate our method's predictive capacity under standard world model benchmarks in Section 5.2. We also show the performance gain under longer planning horizons and lower compute effort, which is crucial in robotic scenarios, in Section 5.3. Furthermore, we introduce two more realistic robot manipulation tasks (Section 5.5) to evaluate Σ-LeWM's end-to-end capability on more challenging tasks.

To answer (RQ2) we perform dimensionality reduction on the learned latent space of Σ-LeWM and investigate whether the physical quantities of the task (e.g., object position) are preserved in the structure of the embedding (Section 5.6). We also learn a GC-IDM [20] planner for both LeWM and Σ-LeWM, evaluating whether the learned latent of Σ-LeWM is easier for other planner methods to perform.

Finally, for (RQ3), we conduct a factorial ablation study on the two auxiliary tasks of backward and inverse dynamics, illustrating that only combining both, i.e., maximising the S-Information, leads to better task performance (Section 5.7).

Figure 2 gives a short overview of the main experimental results, highlighting the performance gains and Σ-LeWM's increased ability to handle high-difficulty tasks common in robotics.

Table 1: **Standard benchmark success (%).** All methods use the LeWM CEM planner setting ($h$=5, CEM30). Our measurements are over 3 training seeds × 5 evaluation seeds × 50 episodes. ‡One of three training seeds collapses (20.0); the other two reach 99.6. †LeWM trained with our multi-step forward term only ($\lambda_{\text{back}} = \lambda_{\text{inv}} = 0$), which is the "base" arm of Section 5.7. §Σ-LeWM with the backward term aligned directly to the raw latent $z_t$ (no projection head).

| | Push-T | TwoRoom | Reacher | Cube | Scene |
|---|---|---|---|---|---|
| Random policy (floor) | 2.0 ±2.0 | 23.6 ±3.6 | 13.6 ±3.8 | 43.6 ±8.9 | 43.2 ±6.7 |
| LeWM [9] | 90.0 ±5.7 | 90.9 ±4.1 | **86.0** ±6.3 | 66.5 ±6.3 | 57.7 ±8.0 |
| LeWM + multi-step† | 88.9 ±3.3 | 92.8 ±4.2 | 83.6 ±4.4 | 71.0 ±5.3 | 53.6 ±5.9 |
| Sub-JEPA [16] | 84.0 ±8.7 | **93.6** ±2.8 | 80.7 ±5.9 | 70.3 ±6.6 | 55.5 ±8.1 |
| RC-aux [17] | 87.9 ±5.2 | 91.6 ±3.4 | 84.8 ±5.4 | 70.3 ±5.4 | 58.0 ±5.5 |
| SMWM [18] | 88.5 ±5.4 | 73.1‡ ±38.9 | 84.5 ±5.8 | 81.9 ±4.9 | 60.0 ±3.1 |
| AC-MTM [19] | 89.2 ±3.8 | 92.4 ±2.7 | 84.9 ±2.4 | 77.3 ±5.9 | 60.9 ±5.1 |
| Backward to raw $z_t$§ | 81.7 ±6.7 | 87.3 ±5.6 | 83.3 ±5.4 | 71.7 ±7.0 | 76.3 ±4.1 |
| Σ-LeWM (ours) | **94.5** ±3.0 | 92.4 ±4.4 | **86.0** ±4.8 | **82.6** ±5.0 | **80.7** ±4.2 |

### 5.1 Experimental Setup

Across the tasks, we use the same set of auxiliary training parameters in the results unless mentioned, including the weight of the multi-step forward prediction $\lambda_{\text{multi}} = 1.0$ with maximum prediction offset $m = 5$, and the weights with respect to the auxiliary objectives across all the experiments: $\lambda_{\text{back}} = 0.1$ for the backward, and $\lambda_{\text{inv}} = 0.05$ for the inverse; an ablation study on these parameters is shown in Section 5.7. We use DCL with temperature 0.1 and in-batch negatives (batch size 128). Consistent with LeWM and to be comparable, we use the same encoder, predictor, optimiser, training protocols, and set of parameters related to the forward prediction training (learning rate, SIGReg weight $\lambda_{\text{sig}} = 0.09$, temperature of the SIGReg $\tau = 0.1$) and the planning scheme using a CEM solver sampling 300 action sequences per iteration and selecting the top 30 candidates to update the sampling distribution.

### 5.2 Results on the Standard Benchmarks

First, we evaluate our method and baselines on LeWM's four standard benchmarks and OGBench-Scene.

**Task setup.** We use the standard Push-T, TwoRoom, Reacher, OGBench-Cube, and OGBench-Scene [28] tasks and use default training and planning settings in LeWM. We follow LeWM's default test-time planning settings: a CEM solver with 30 optimisation steps, a 5-step planning horizon, and environment timesteps set to 25 with a 5-step frame skip. We compare against LeWM, Sub-JEPA [16] which replaces SIGReg with a subspace Gaussian regulariser, RC-aux [17], which uses two auxiliary objectives for time/space alignment, and SMWM [18] and AC-MTM [19], which add an inverse dynamics loss, in contrastive form for AC-MTM. For LeWM and Sub-JEPA, training seed 3072 is the authors' released checkpoint and seeds 1 and 2 are retrained with the released recipe; RC-aux applies the authors' fine-tuning recipe to each LeWM seed. All baselines keep their original objectives and therefore train without our multi-step forward term. On Scene, which has no released checkpoints, we train every method ourselves. To separate the contribution of the multi-step term from that of the backward and inverse objectives, Table 1 also reports LeWM trained with the multi-step term alone on all five tasks (the "base" arm of Section 5.7). We report the mean and standard deviation over 3 training seeds × 5 evaluation seeds × 50 episodes.

**Results.** Experiments (Table 1) show that Σ-LeWM is best on Scene by about 20 points and on Push-T by 4.5 points; on the other three tasks it is within noise of the best method: TwoRoom (92.4 vs. 93.6 for Sub-JEPA), Cube (82.6 vs. 81.9 for SMWM) and Reacher (86.0 vs. 86.0 for LeWM). The multi-step forward term alone stays close to LeWM (Push-T 88.9, TwoRoom 92.8, Reacher 83.6, Cube 71.0, Scene 53.6), so the gains of Σ-LeWM on Scene (80.7) and Cube come from the backward and inverse objectives.

It is worth noting that TwoRoom is a relatively low-complexity task, in which a point-mass has to navigate a gap.

### 5.3 Results on Extended Horizons

Many real-world tasks, especially in robotics, benefit from long planning horizons. In these experiments we analyse further Σ-LeWM's behaviour under long-horizon planning.

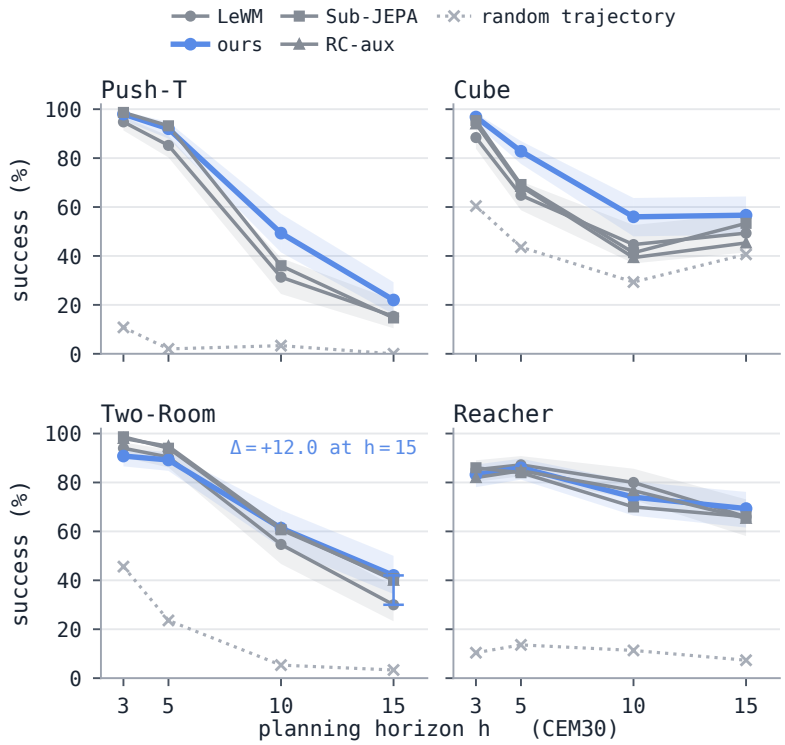



Figure 3: **Longer horizons**, at fixed inference effort CEM30 on all four tasks. $\Delta$: the performance gain over LeWM.

**Task setup.** We extend the planning horizon from $h = 5$ to $h = 3, 5, 10, 15$ to test robustness to longer lookahead planning, which is especially crucial for robot learning and planning. We compare against LeWM, Sub-JEPA, and RC-aux with $h = 3, 5, 10, 15$ horizons using the four standard benchmarks above.

**Results.** We show performance gains over all baselines on Push-T and Cube tasks across horizon choices, as shown in Fig. 3. Notably, we can observe a performance gain when extending the horizon to $h = 15$ on TwoRoom and Reacher tasks, demonstrating the long-term planning capability of our method, which is essential for robot learning tasks.

### 5.4 Results on Planning Efficiency

Planning with less compute is another crucial aspect for robot learning and planning.

**Task setup.** We therefore arrange a planning efficiency experiment with the CEM solver, changing its budget during test-time inference, where the number of iteration steps changes to $n_{\text{CEM}} = 8, 4, 2$ and therefore the CEM search budget is reduced by $\times 3.75$, $\times 7.5$ and $\times 15$ (the $x$-axis of Figure 4) compared to the standard $n_{\text{CEM}} = 30$. We compare against the same baselines in Section 5.3.

**Results.** We show in Fig. 4 that our method outperforms the baselines with substantially less search on Push-T and Cube. On Push-T, our method uses 8 CEM iterations to outperform LeWM at 30 CEM iterations (91.2% vs. 85.2%), and only 4 CEM iterations are already better than LeWM at 8 (Figure 4) and remain competitive with LeWM at 30 while using a $\times 7.5$ smaller CEM search budget. Thus the gain is not only in the final success rate, but also in the test-time planning compute required to reach it.

### 5.5 Robotic Tasks: Gear-Assembly and Cube-Stacking

To evaluate if our method has an advantage on more realistic tasks, we introduce two manipulation tasks with strict success criteria.

**Task setup.** We arrange the Cube-Stacking and Gear-Assembly tasks based on IsaacSim simulation environments [29] shown in Figure 6. The Cube-Stacking benchmark is derived from augmented teleoperation trajectories [30] (1k trajectories). The Gear-Assembly dataset is collected by a learned policy from SRSA [31] with 10k trajectories, each

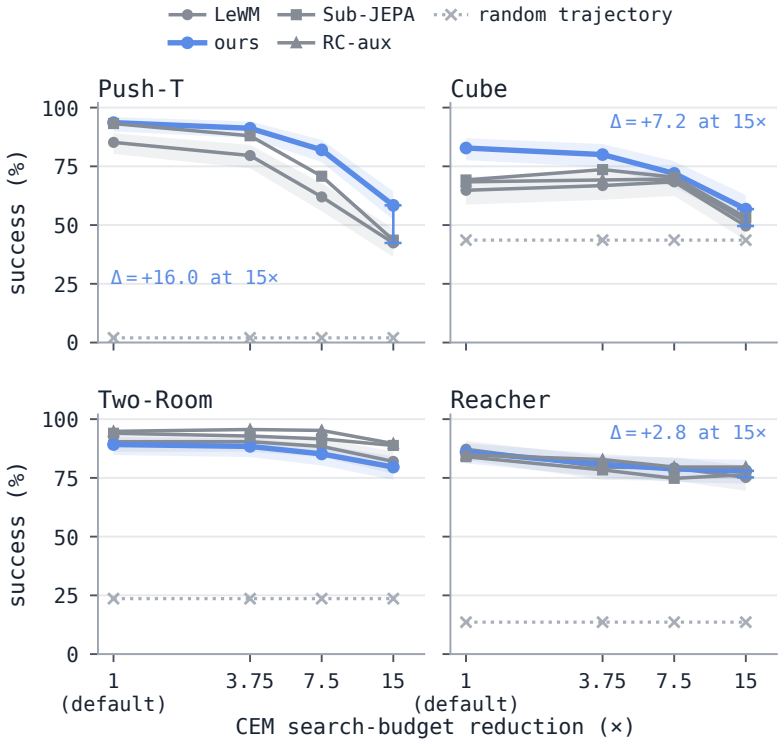



Figure 4: **Planning efficiency** with planning horizon $h$=5; the $x$-axis is the search reduction relative to LeWM's default planning budget CEM30. Moving right corresponds to lower test-time planning compute.

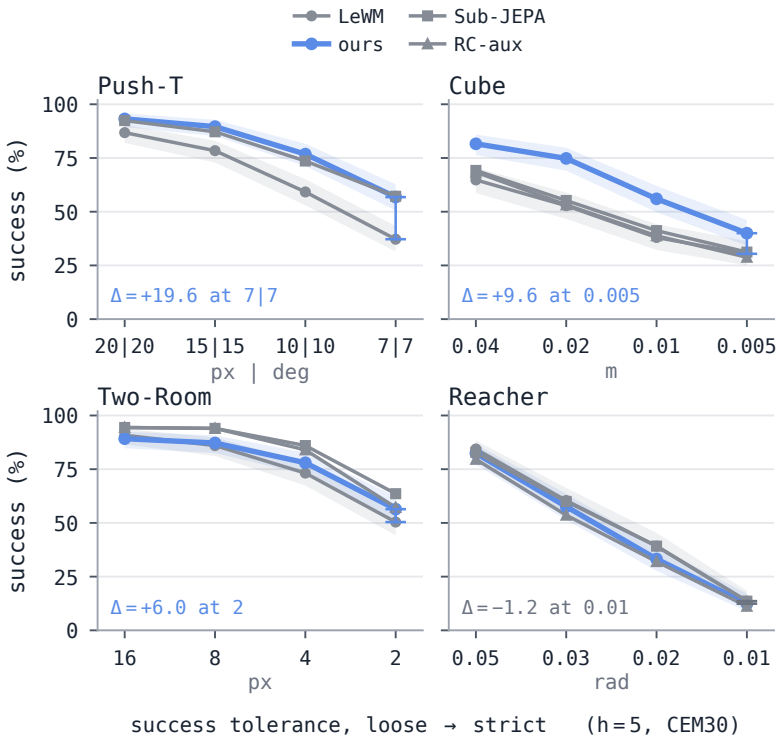



Figure 5: **Task difficulty.** Σ-LeWM outperforms LeWM under increasing task difficulty, where the success criterion is increasingly narrowed for goals (x-axis).

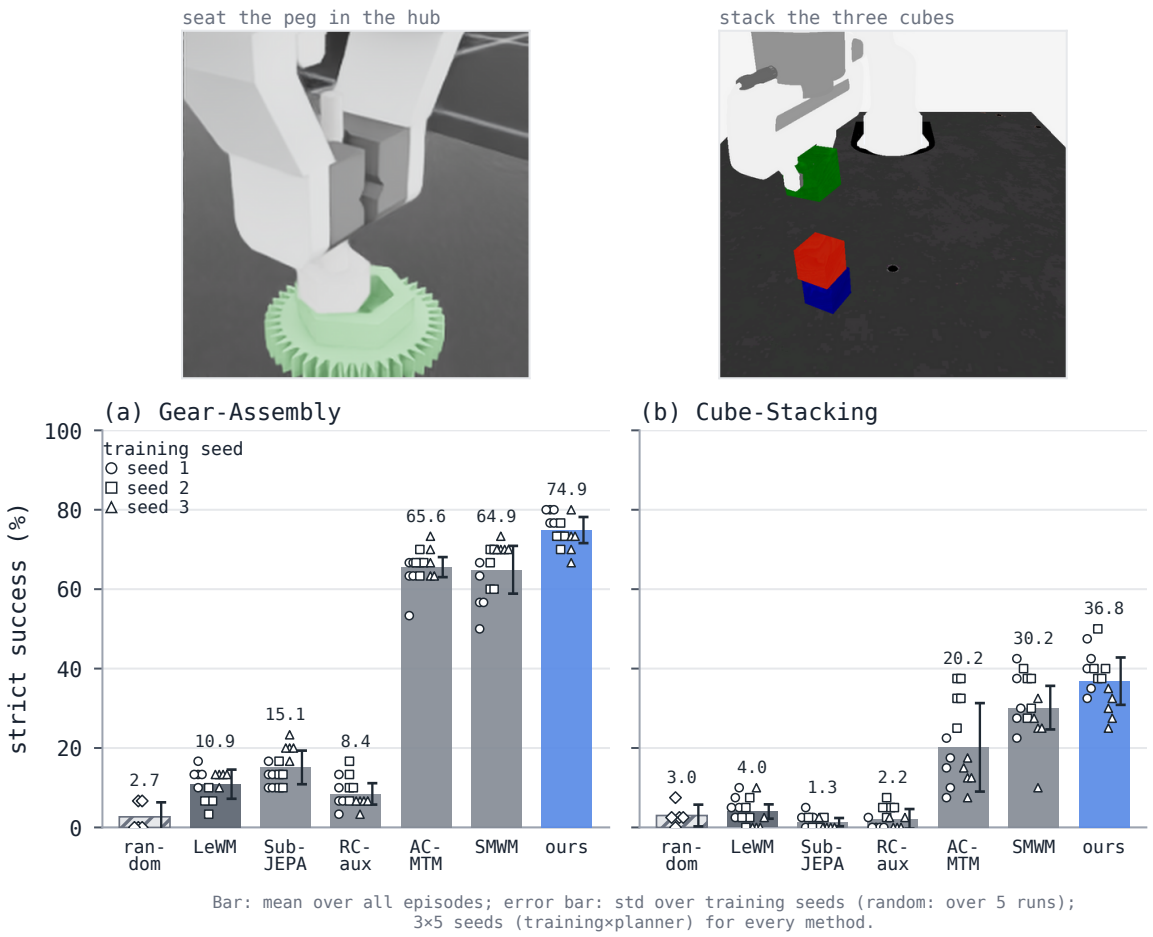



Figure 6: Comparison of Σ-LeWM (ours) versus the baselines on robotic manipulation tasks: Gear-Assembly and Cube-Stacking.

with 75 environment steps. We further filter the trajectories for strict success in terms of assembly and reduce the dataset to around 21k frames. We use $\lambda_{\text{sig}} = 0$ for both Gear-Assembly and Cube-Stacking, and the others remain the same. We compare against LeWM, Sub-JEPA, RC-aux, and baselines with additional dynamics terms, where SMWM [18] utilises an inverse dynamics loss to avoid latent collapse, and AC-MTM [19] shares the same inspiration but uses a contrastive version of it. At test time, we use the first frame that satisfies the task-success criterion as the goal frame from the test set whose episodes are excluded from training, rather than relying on the fixed look-ahead goals used in the standard benchmarks. We use horizon $h = 5$ for Gear-Assembly and $h = 10$ for Cube-Stacking.

**Results.** We demonstrate (Fig. 6) that our method improves strict success from 10.9% for LeWM to 74.9%, and also outperforms the inverse-dynamics baselines on Gear-Assembly. Section 5.6 examines one candidate explanation: whether the task-success state is linearly decodable from the frozen representation.

### 5.6 Quality of the Latent Representation

We conduct two different analyses to illuminate the quality and structure of the learned embedding by our method: physical interpretation, and planner learnability.

**Physical interpretation.** We show the principal component analysis (PCA) visualisation for the latent representation and highlight the specific physical quantities to illustrate if the learned latent reflects the distribution of a certain quantity. In Figure 7, the first four columns use PCA projections of the frozen latents, while Gear-Assembly in the last column uses a supervised linear view to distinguish whether the binary success signal is captured in the latent. Each column represents the distribution of a certain physical quantity in different tasks, coloured by the value of the quantity. Appendix A quantifies them with held-out linear probes.

**Planner learnability.** From the robot learning perspective, we also demonstrate the planner learnability on the latent representation to further evaluate more practically. Essentially, we illustrate that our method, combined with a

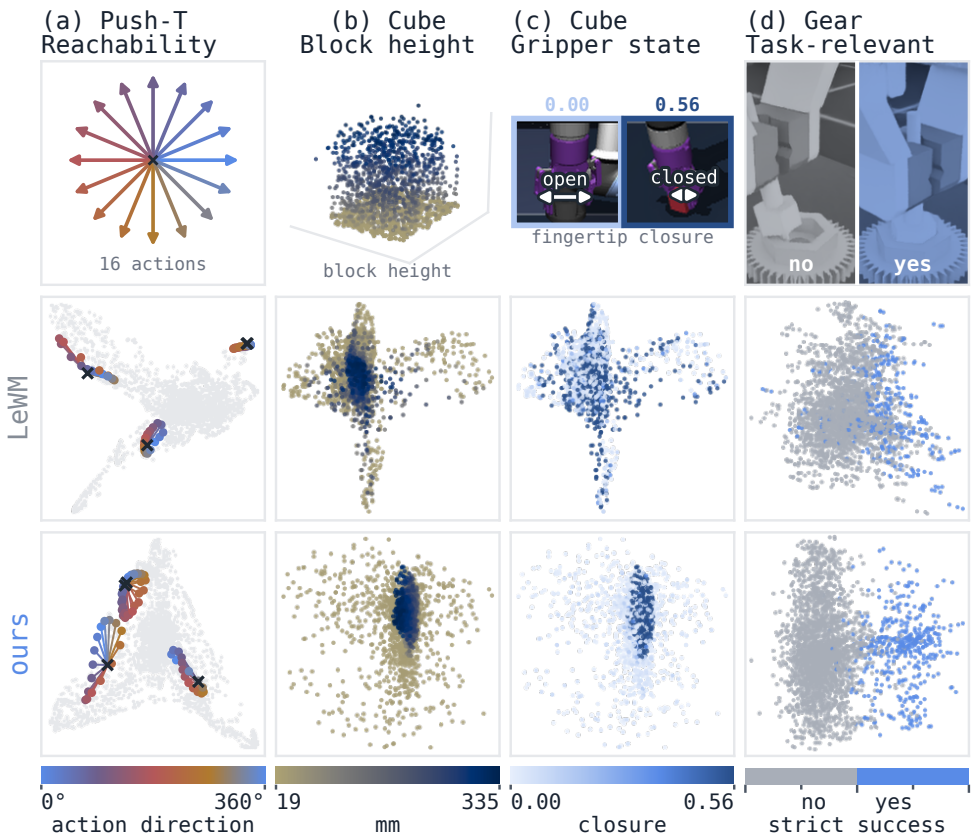



Figure 7: **Physical interpretation, LeWM vs Σ-LeWM**. PCA visualisations of each model's latent space, coloured by physical quantities of the tasks, showing explainable latent distributions. (a). Push-T 1-step reachability in the latent space from a fixed latent point $\mathbf{z}_t$ under sixteen normalised 2-D action directions, showing how far and in which directions $f(\mathbf{z}_t, \tilde{\mathbf{a}}_t)$ can move. (b). For OGBench-Cube block height, our representation shows a clear height-dependent structure, with high-block states concentrated in a compact region. (c). The gripper state in OGBench-Cube, with our representation showing closed-gripper states concentrated in a compact region, which LeWM separates. (d). For Gear, task-success states form a clearly separated latent region under ours, while LeWM shows a mixture between success and failure.

state-of-the-art planner [20], can further obtain performance gains compared to learning the planners on other latent representations. We train the same GC-IDM [20] planner model separately on LeWM and our method and compare the OGBench-Cube performance. We show in Figure 8 that with our latent representation, a learned planner performs better than the one on LeWM.

### 5.7 Ablation Study: Why Backward/Inverse are Necessary

**Ablation on the objective types.** We conduct factorial ablations on the effectiveness of the introduced contrastive tasks for the backward and inverse dynamics. Specifically, we investigate the single-term effectiveness by keeping two terms of the proposed contrastive objective, or instead, the regression version. We show in Figure 9 that both backward and inverse objectives gain performance on OGBench-Cube, while combining them leads to the best performance. In addition, using the regression objective (MSE) instead of the contrastive version in both cases results in a performance drop, indicating that the contrastive objectives capture better structure for the latent representation in terms of the final performance.

**Ablation on the objective weights.** We scale one loss term for each training on Push-T with horizon $h$=5 while the other two remain as the default ($\lambda_{\text{sig}} = 0.09$, $\lambda_{\text{back}} = 0.1$, and $\lambda_{\text{inv}} = 0.05$). We find that this set of weights is better in both scaling directions for every term (Table 2). This sweep uses evaluation seeds different from those of Table 1.

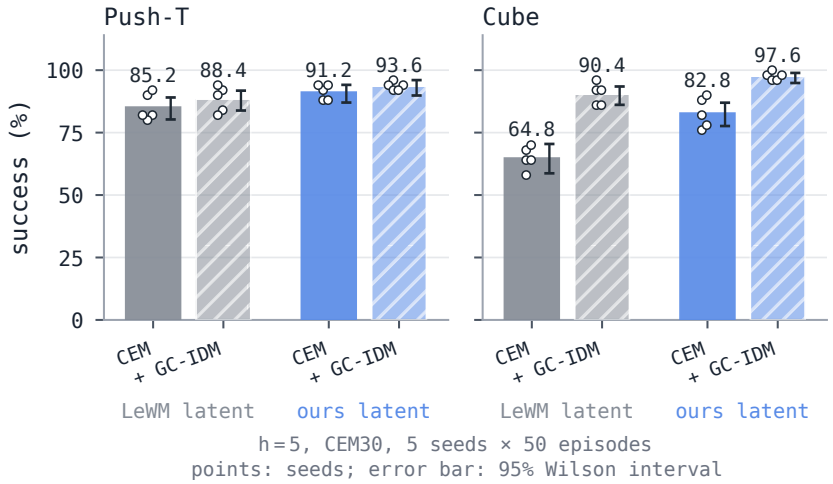



Figure 8: **Planner learnability, LeWM vs Σ-LeWM.** Performance with default CEM planner and trained GC-IDM [20] planner on Push-T (left) and OGBench-Cube (right). The GC-IDM planners are trained on the learned latent representation with frozen encoders, and tested with 5 seeds × 50 episodes for each bin.

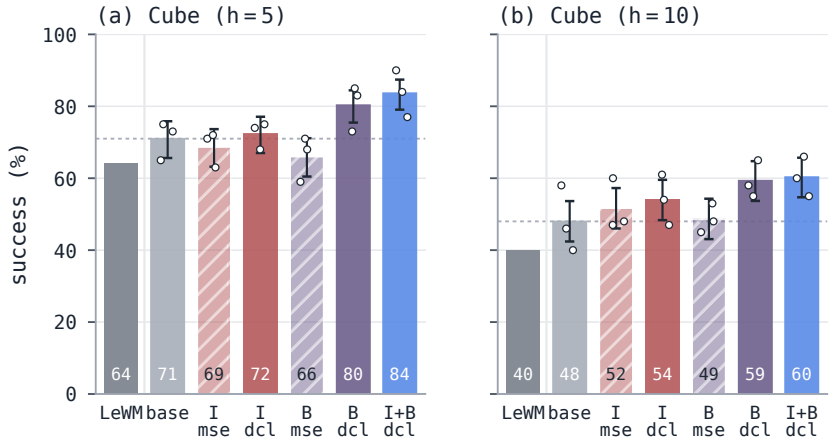



Figure 9: Comparison of Σ-LeWM versus standard LeWM and ablations of different combinations of loss functions. On the x-axis, *base* keeps only the multi-step forward objective ($\lambda_{\text{back}} = \lambda_{\text{inv}} = 0$; dashed line); *I* and *B* add the inverse or the backward term alone, trained with a regression (*mse*) or contrastive (*dcl*) loss, and *I+B dcl* is the full Σ-LeWM. Evaluations with both short (h=5) and longer (h=10) horizon planning demonstrate the effectiveness and performance gain of Σ-LeWM when reflecting the S-Information. This ablation uses its own evaluation seeds (three banks of 100 episodes per checkpoint), so its values differ slightly from Table 1.

Table 2: Loss-weight ablation on Push-T: $h$=5 success (%) when one term's weight is scaled while the other two stay frozen (1× are the weights used everywhere else in the paper).

| loss weight multiplier | 0.5× | 1× | 2× |
|---|---|---|---|
| inverse $\mathcal{L}_{\text{inv.}}$ | 82 | **90** | 87 |
| backward $\mathcal{L}_{\text{back.}}$ | 86 | **90** | 82 |
| SIGReg $\mathcal{L}_{\text{SIG}}$ | 88 | **90** | 76 |

## 6 Conclusion

We propose a new set of auxiliary tasks to enable the learning of robust and well-structured latent embeddings in world models such as LeWorldModel. Our proposed loss-functions are based on insight from information theory, and complement the standard objectives of LeWM in a theory-grounded way. Adding loss functions for backward and inverse dynamics in the latent space is equivalent to maximising the Dual and Total Correlations, in short the S-Information. This approach leads to an increase in determinism, or predictability, between the latent variables while rewarding their alignment at the same time. Experiments demonstrated that the proposed method Σ-LeWM shows strong performance improvements over LeWM, especially in robot manipulation benchmarks. Furthermore, we note an increased stability for long-horizon planning, an important feature for robotic tasks in the real world. Future work will investigate the use of the current auxiliary tasks, the backward and inverse latent dynamics, at planning time and the deployment of Σ-LeWM on real-world robot tasks.

## References


[1] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017.

[2] P. Lancaster, N. Hansen, A. Rajeswaran, and V. Kumar, "Modem-v2: Visuo-motor world models for real-world robot manipulation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[3] S. Ye, Y. Ge, K. Zheng *et al.*, "World action models are zero-shot policies," *arXiv preprint arXiv:2602.15922*, 2026.

[4] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta, "Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets," *arXiv preprint arXiv:2504.02792*, 2025.

[5] Y. Huang, C. Agia, J. Wu, T. Hermans, and J. Bohg, "Points2plans: From point clouds to long-horizon plans with composable relational dynamics," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 1208–1216.

[6] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations*, 2020.

[7] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[8] N. A. Hansen, H. Su, and X. Wang, "Temporal difference learning for model predictive control," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 17–23 Jul 2022, pp. 8387–8406.

[9] L. Maes, Q. L. Lidec, D. Scieur, Y. LeCun, and R. Balestriero, "Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels," *arXiv preprint arXiv:2603.19312*, 2026.

[10] D. Hafner *et al.*, "Learning latent dynamics for planning from pixels," in *International Conference on Machine Learning (ICML)*. PMLR, 2019.

[11] E. Alonso *et al.*, "Diffusion for world modeling: Visual details matter in atari," in *Advances in Neural Information Processing Systems 37*, 2024, pp. 58 757–58 791.

[12] M. Assran, Q. Duval, I. Misra *et al.*, "Self-supervised learning from images with a joint-embedding predictive architecture," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2023.

[13] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus *et al.*, "V-jepa 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.

[14] G. Zhou, H. Pan, Y. Lecun, and L. Pinto, "DINO-WM: World models on pre-trained visual features enable zero-shot planning," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025.

[15] U. Sobal, W. Zhang, K. Cho, R. Balestriero, T. G. Rudner, and Y. LeCun, "Learning from reward-free offline data: A case for planning with latent dynamics models," *Advances in Neural Information Processing Systems*, vol. 38, pp. 43 905–43 941, 2026.

[16] K. Zhao, D. Nie, Y. Lin, Z. Luo, Y. Gu, D.-P. Fan, and D. Zeng, "Sub-jepa: Subspace gaussian regularization for stable end-to-end world models," *arXiv preprint arXiv:2605.09241*, 2026.

[17] W. Li, G. Li, K. Maeda, T. Ogawa, and M. Haseyama, "Predictive but not plannable: Rc-aux for latent world models," *arXiv preprint arXiv:2605.07278*, 2026.

[18] P. Ivashkov, R. Balestriero, and B. Schölkopf, "Sensorimotor world models: Perception for action via inverse dynamics," *arXiv preprint arXiv:2606.20104*, 2026.

[19] J. Boylan and C. Hokamp, "No gaussian required: Contrastive inverse dynamics for jepa world models," *arXiv preprint arXiv:2608.17542*, 2026.

[20] H. Nguyen, X. Xu, and X. Huang, "Latent geometry beyond search: Amortizing planning in world models," *arXiv preprint arXiv:2605.08732*, 2026.

[21] G. Seo, D. Kim, and S. Kwak, "Acid: Action consistency via inverse dynamics for planning with world models," *arXiv preprint arXiv:2607.02403*, 2026.

[22] C.-H. Yeh, C.-Y. Hong, Y.-C. Hsu, T.-L. Liu, Y. Chen, and Y. LeCun, "Decoupled contrastive learning," in *European Conference on Computer Vision (ECCV)*. Springer, 2022, pp. 668–684.

[23] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[24] B. Poole, S. Ozair, A. Van Den Oord, A. Alemi, and G. Tucker, "On variational bounds of mutual information," *Proceedings of the 36th International Conference on Machine Learning*, vol. 97, 2019.

[25] S. Watanabe, "Information theoretical analysis of multivariate correlation," *IBM Journal of research and development*, vol. 4, no. 1, pp. 66–82, 1960.

[26] H. Te Sun, "Nonnegative entropy measures of multivariate symmetric correlations," *Information and Control*, vol. 36, pp. 133–156, 1978.

[27] F. E. Rosas, P. A. Mediano, M. Gastpar, and H. J. Jensen, "Quantifying high-order interdependencies via multivariate extensions of the mutual information," *Physical Review E*, vol. 100, no. 3, p. 032305, 2019.

[28] S. Park, K. Frans, B. Eysenbach, and S. Levine, "Ogbench: Benchmarking offline goal-conditioned rl," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 94 937–94 982.

[29] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar *et al.*, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[30] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," in *7th Annual Conference on Robot Learning*, 2023.

[31] Y. Guo, B. Tang, I. Akinola, D. Fox, A. Gupta, and Y. Narang, "Srsa: Skill retrieval and adaptation for robotic assembly tasks," in *International Conference on Learning Representations*, vol. 2025, 2025.

Table 3: **Linear probes on OGBench-Cube** (1800 frames from 300 unseen episodes). Test score, mean ± sd over 5 episode-grouped splits.

| | Block height $R^2$ | Gripper closed AUROC | Gripper closed Acc. | Gripper opening $R^2$ | Effector $xyz$ $R^2$ | Block $xy$ $R^2$ | Effector−block $R^2$ |
|---|---|---|---|---|---|---|---|
| LeWM | 0.958 ±0.006 | **0.988** ±0.005 | 0.950 ±0.012 | 0.837 ±0.035 | 0.837 ±0.023 | 0.929 ±0.030 | 0.775 ±0.055 |
| LeWM + multi-step | 0.951 ±0.012 | 0.986 ±0.004 | 0.951 ±0.011 | 0.838 ±0.026 | 0.980 ±0.000 | 0.647 ±0.037 | 0.728 ±0.025 |
| Sub-JEPA | 0.755 ±0.022 | 0.980 ±0.005 | 0.944 ±0.005 | 0.761 ±0.026 | 0.929 ±0.005 | 0.898 ±0.013 | 0.887 ±0.006 |
| RC-aux | 0.958 ±0.005 | **0.988** ±0.006 | 0.954 ±0.009 | 0.840 ±0.035 | 0.843 ±0.026 | **0.933** ±0.026 | 0.783 ±0.062 |
| Σ-LeWM | **0.978** ±0.005 | **0.988** ±0.007 | **0.963** ±0.014 | **0.888** ±0.027 | **0.997** ±0.000 | 0.846 ±0.016 | **0.900** ±0.014 |

Table 4: **Measured planning latency of the CEM ladder** for a single-environment MPC solve, $h$=5, 300 candidates, on two GPUs. Mean ± sd over the four standard tasks and over LeWM, Sub-JEPA, RC-aux and Σ-LeWM, whose planners are identical at test time. Replanning rate and speed-up include observation preprocessing (≈1 ms); latency does not.

| | CEM30 | CEM8 | CEM4 | CEM2 |
|---|---|---|---|---|
| Nominal budget reduction | ×1 | ×3.75 | ×7.5 | ×15 |
| *NVIDIA GeForce RTX 3090* | | | | |
| Solve latency (ms) | 692.2 ±7.9 | 191.0 ±1.9 | 99.5 ±0.9 | 53.8 ±0.7 |
| Replanning rate (Hz) | 1.44 ±0.02 | 5.21 ±0.05 | 9.97 ±0.09 | 18.31 ±0.22 |
| Measured speed-up | ×1.00 | ×3.61 ±0.02 | ×6.91 ±0.05 | ×12.69 ±0.10 |
| *NVIDIA H100* | | | | |
| Solve latency (ms) | 450.9 ±3.6 | 127.0 ±1.2 | 68.2 ±0.7 | 38.6 ±0.6 |
| Replanning rate (Hz) | 2.21 ±0.02 | 7.82 ±0.07 | 14.48 ±0.14 | 25.30 ±0.32 |
| Measured speed-up | ×1.00 | ×3.53 ±0.01 | ×6.54 ±0.03 | ×11.43 ±0.08 |

## A Quantitative Probes of the Frozen Representation

To complement the qualitative PCA views of Figure 7, we measure how much simulator state a linear probe recovers from the frozen latent representation on OGBench-Cube.

**Task setup.** We freeze all the encoders, and all frames come from fresh episodes unseen by every encoder. We fit ridge regression, or logistic regression for binary targets, on standardised features, using episode-grouped 60/20/20 splits that are shared by all encoders and repeated with five seeds.

**Results.** We find that the two terms play different roles (Table 3). We observe that the multi-step forward term improves the robot's own state, increasing the end-effector accuracy from $0.84$ to $0.98$, but that it does so at the cost of object information: the block's planar position falls from $0.93$ to $0.65$. However, adding the backward and inverse terms recovers most of this loss while preserving the improvement in end-effector representation. It also improves the representation of the relative position between the end-effector and the block compared with either alternative. We also find that the gripper state is already saturated under LeWM.

## B Measured Planning Latency

Section 5.4 reduces the CEM search budget, which counts optimisation iterations rather than time; we therefore measure the latency of the planner itself.

**Task setup.** We evaluate the inference time of the standard single-environment MPC solver on a single desktop GeForce RTX 3090 and, as a data centre reference, a single NVIDIA H100, with 5 warm-up solves and 50 repetitions per setting.

**Results.** We find that solve time grows linearly with the number of CEM iterations, almost all of it spent by the predictor rolling out candidate plans (Table 4). We observe that cutting CEM30 to CEM4 makes a solve about seven times faster on both GPUs. We also find that the H100 is only about $1.5\times$ faster than the GeForce RTX 3090, since a ViT-tiny predictor scoring 300 candidates is latency-bound rather than compute-bound. All four methods time within two percent of each other, because the auxiliary heads are discarded after training.